



Movie Recommendation System

Big Data Project

Hybrid Recommendation Engine using PySpark, ALS & LSH

Team - ScaleCraft

BY

NAGARAJA S B

SANUJ KULSHRESTHA

THANGARAJ N

SWAPNA GUNDEBOYINA

Problem Definition

- ▶ Personalized movie recommendations at scale (25M+ ratings).
 - ▶ Low latency (<3 seconds) and high accuracy ($\text{RMSE} < 0.85$).
 - ▶ Cold-start handling for new users.



Problem Motivation

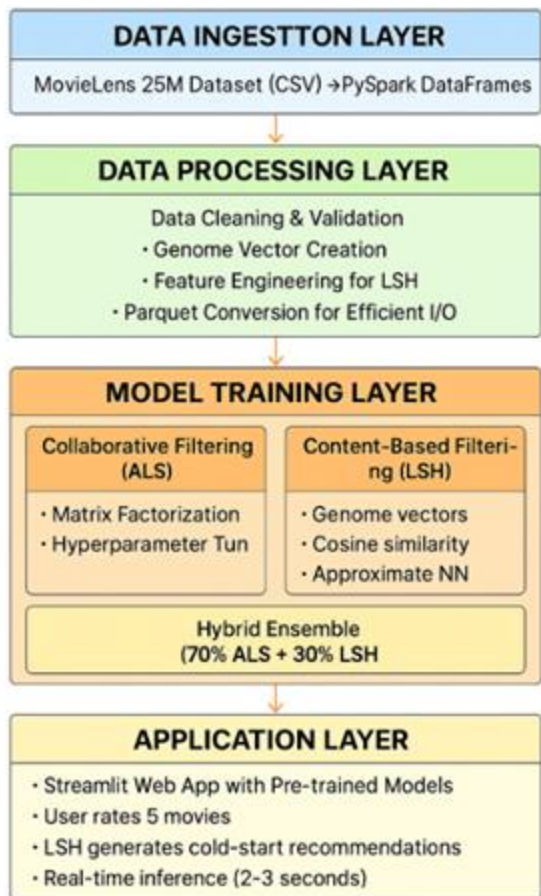
- ▶ Recommendations drive 80% of content watched.
- ▶ Inappropriate Recommendations may lead to new user attrition.
- ▶ Processing 25M ratings requires distributed computing.
- ▶ Real-time responses required under 3 seconds.
- ▶ Can be extended to other recommendation systems (e-commerce, Advertising, Music)

Design Goals

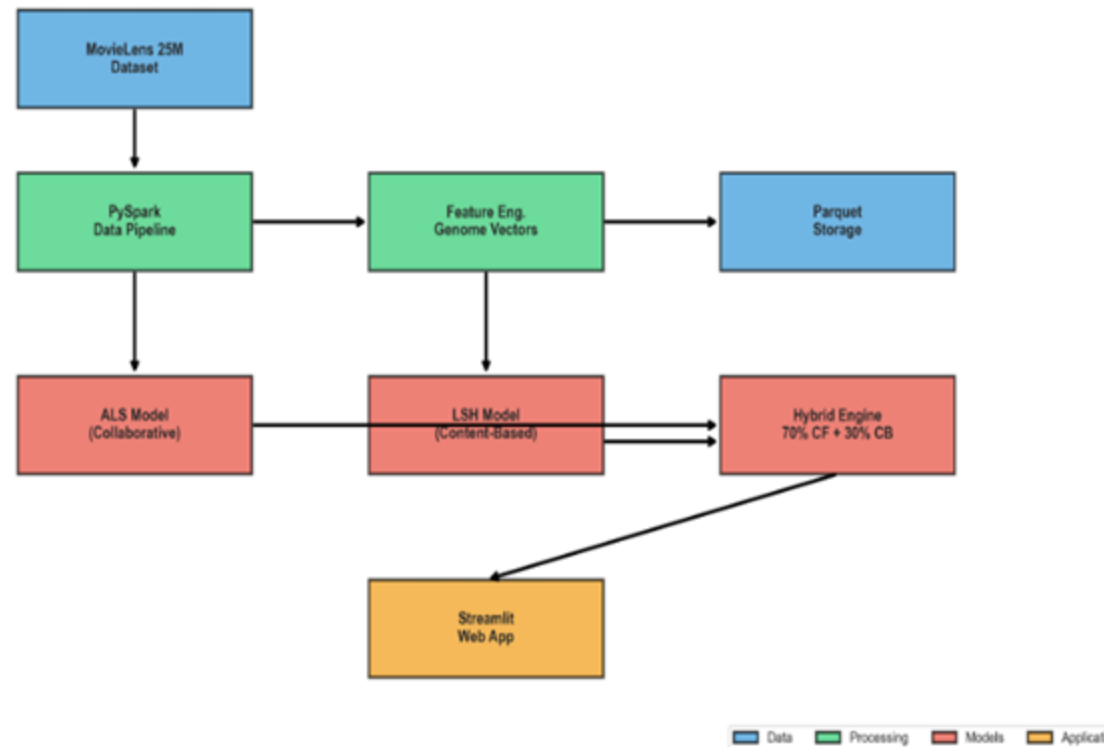
- ▶ High accuracy.
- ▶ Scalability.
- ▶ Low latency inference pipeline.
- ▶ Cold-start solution for new user recommendations.
- ▶ Production-ready reusable models.

.

Data Pipeline Architecture



Movie Recommendation System Architecture



Model - ALS

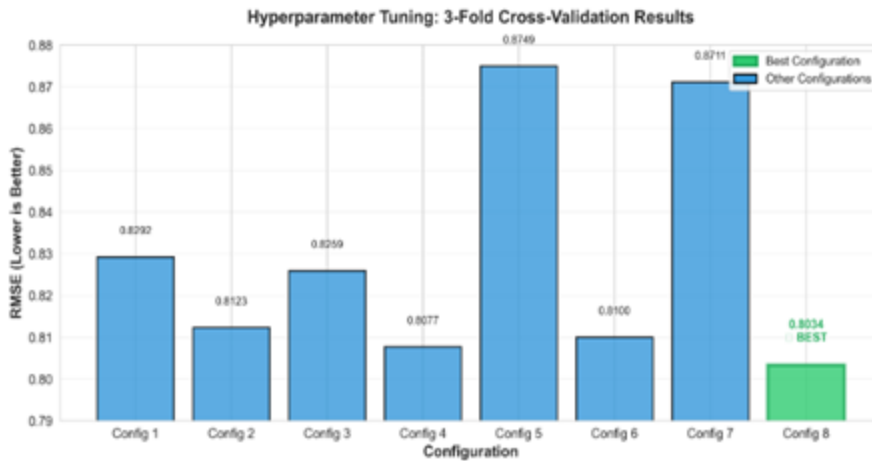
ALS (Alternating Least Squares)

- ▶ Matrix factorization: $R \approx U \times V^T$.
- ▶ Decomposes user-item rating matrix into two lower-rank matrices:
 - ▶ User factors (162,541 users $\times$ rank)
 - ▶ Item factors (62,423 movies $\times$ rank)
- ▶ Learns User Preferences
- ▶ Discovers hidden patterns
- ▶ 3-fold cross-validation (8 configurations)
- ▶ Best Parameters: (Rank – 30, Max Iterations – 20, Regularization – 0.1)

Why ALS over other CF Algos ?

Algorithm	Pros	Cons	Our Choice
ALS (Alternative Least Squares)	Handles sparse data Parallelizable Implicit feedback support Proven at scale (Netflix, Spotify)	Cold-start problem	CHOSEN
SVD (Singular Value Decomposition)	Accurate	Not parallelizable Requires dense matrix	✗
KNN (K Nearest Neighbors)	Simple	$O(n^2)$ complexity Doesn't scale	✗
Neural CF	State-of-art	Requires GPU Complex tuning Overkill for this dataset	✗

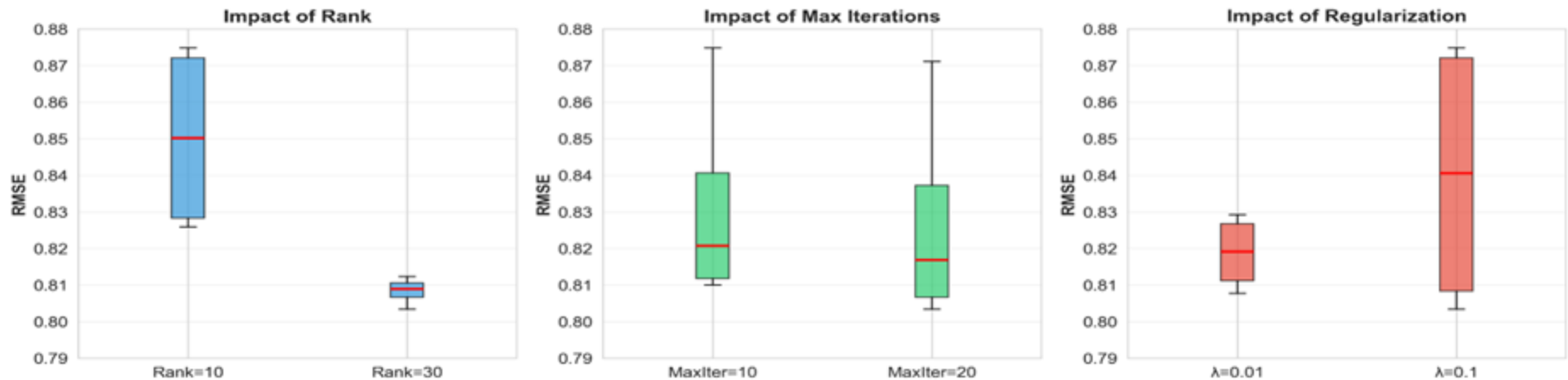
ALS Hyperparameter Tuning



Config	Rank	Max Iter	Reg Param	RMSE
1	10	10	0.01	0.8292
2	10	10	0.1	0.8123
3	10	20	0.01	0.8259
4	10	20	0.1	0.8077
5	30	10	0.01	0.8749
6	30	10	0.1	0.8100
7	30	20	0.01	0.8711
8	30	20	0.1	0.8034

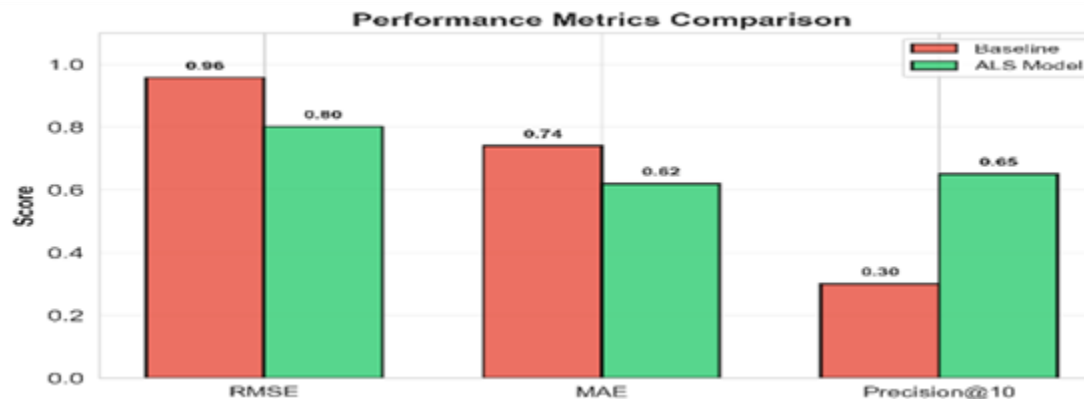
Hyperparameter Impact on Model Performance

Hyperparameter Impact on Model Performance



ALS vs Baseline Model

Metric	Baseline	Our Model	Improvement
RMSE	0.9571	0.8011	16.30% ↓
MAE	0.7398	0.6195	16.27% ↓
Precision@10	0.30	0.65	65% accuracy
Training Time	0s	~120-150 min	Acceptable with single machine
Inference Time	<1s	2-3s	Fast



Model - LSH

LSH (Locality Sensitive Hashing)

- ▶ Approximate nearest neighbor search algorithm
 - ▶ Maps similar items to same "buckets" using hash functions
 - ▶ Enables fast similarity search in high-dimensional spaces
 - ▶ Random Projections from 1128-d to lower dim space.
 - ▶ No User History Needed.
 - ▶ Used for Cold-Start Problem Handling

 - ▶ Parameters:
 - ▶ Bucket Length: 2
 - ▶ Controls granularity. Balances precision vs recall.
 - ▶ Hash Tables: 3
 - ▶ Each Hash Table uses different random projections.
 - ▶ Dimensions: 1128
1. **Random Projection:**
 - Generate 3 random hyperplanes in 1,128-dim space
 - Each hyperplane divides space into 2 regions
 2. **Hashing:**
 - For each movie vector, determine which side of hyperplane
 - Create binary hash code (e.g., [1, 0, 1])
 3. **Bucketing:**
 - Movies with same hash code → same bucket
 - Similar movies likely in same bucket
 4. **Similarity Search:**
 - Query movie's bucket
 - Return nearby movies (approximate neighbors)

Model - Hybrid Recommendation Engine

Approach	Strengths	Weaknesses
Collaborative Filtering (ALS)	Discovers hidden patterns Personalized to user behavior No domain knowledge needed	Cold-start problem Popularity bias Sparsity issues
Content-Based (LSH)	No cold-start Explainable Works for new items	Limited serendipity Over-specialization Requires features
Hybrid (70% CF + 30% Content) $\text{hybrid_score} = 0.7 \times \text{cf_score} + 0.3 \times \text{content_score}$	Best of both worlds Handles cold-start Diverse recommendations	More complex

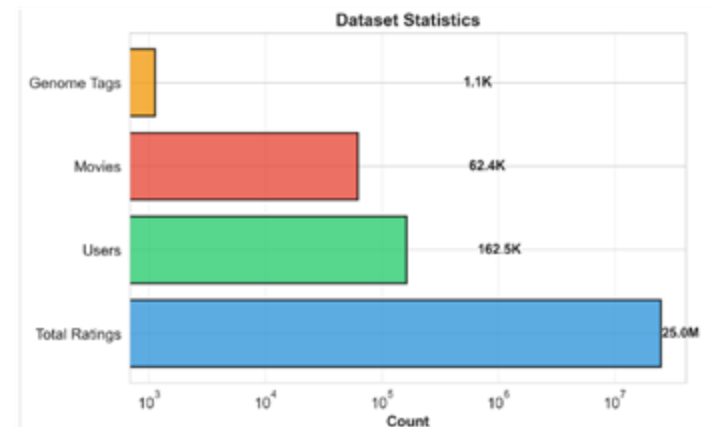
Big Data Platform

Layer	Technology	Purpose
Data Processing	PySpark 4.0.1	Distributed processing of 25M records
Storage	Parquet	Columnar storage for efficient I/O
ML Framework	PySpark MLlib	ALS, LSH, model persistence
Web Framework	Streamlit 1.50.0	Interactive UI
Runtime	Java 17	Required for PySpark 4.x
Language	Python 3.9+	Core development

Scalability & Performance Goals

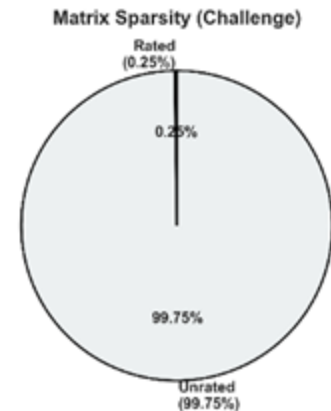
Scalability:

- 25M ratings processed
- 162K users handled
- 62K movies indexed
- 99.75% sparsity



Efficiency:

- Parquet: 60% smaller
- Distributed processing
- Pre-trained models for Instant Recommendation



Scalability & Performance Metrics

Processing Performance:

Task	Data Size	Processing Time	Hardware
Data Loading (CSV → PySpark)	1.1GB	45 sec	8GB RAM, 2 cores
Genome Vector Creation	15M records	12 min	Distributed Spark
ALS Training (10 iterations)	25M ratings	120-150 min	8GB RAM, 2 cores
LSH Model Training	62K items	3 min	Single node
Parquet Conversion	1.1GB → 400MB	8 min	Distributed I/O

Scalability & Performance Metrics

Inference Performance (New Use Recommendation)

Operation	Latency	Method
Load Pre-trained Models	1.2 sec	Disk → Memory
LSH Cold-Start (5 ratings)	2.1 sec	Approximate NN
Total Recommendation Time	2.4 sec	End-to-end

Challenges & Solutions

- ▶ **Cold-Start Problems:**
 - ▶ LSH content-Based Filtering
- ▶ **25M Records:**
 - ▶ PySpark Distributed Computing
- ▶ **99.75% Sparsity:**
 - ▶ ALS + Normalization

Live Demo

- ▶ Interactive Streamlit Web Application
- ▶ User rates 5 movies (e.g., Inception, Interstellar, Matrix)
- ▶ System generates 5 personalized recommendations
- ▶ Response time: 2-3 seconds
- ▶ Demonstrates cold-start handling with LSH

Conclusion & Future Scope

Achievements:

- 16.3% improvement
- Handles 25M ratings
- Solves cold-start
- Production-ready
- Fast inference (<3s)

Key Learnings:

- Hybrid > Pure approach
- PySpark scales well

Future Enhancements:

- Deep learning (NCF)
- Automated Regular Model Updates
- Real-time Learning
 - Context-aware recs (time, day)
 - Multi-objective opt. (watch time)
 - Explainability (why ?)

Production Path:

- A/B testing framework
- Real-time updates

Reference Links

- Github Repository: [Movie Recommender-By-ScaleCraft](#)
- Project Report: [ScaleCraft Project Report](#)
- ALS: [Collaborative Filtering for Implicit Feedback datasets](#)
- LSH: [Locality Sensitive Hashing Scheme on p-stable distribution](#)
- Movie Lens: [25M Movie Ratings Dataset](#)

UI App Screenshots

Movie Recommendation Engine - By ScaleCraft

Rate 5 movies and get personalized recommendations!

 Cold-Start Optimized with LSH

 Loaded pre-trained LSH model from notebook!

Step 1: Select and Rate 5 Movies

Movie 1

Matrix, The (1999)

 Avg: 4.15 |  72,674 ratings |  Action|Sci-Fi|Thriller



Movie 2

Jurassic Park (1993)

 Avg: 3.68 |  64,144 ratings |  Action|Adventure|Sci-Fi|Thriller



Movie 3

Toy Story (1995)

 Avg: 3.89 |  57,309 ratings |  Adventure|Animation|Children|Comedy|Fantasy



Movie 4

Terminator 2: Judgment Day (1991)

 Avg: 3.95 |  57,379 ratings |  Action|Sci-Fi



Movie 5

American Beauty (1999)

 Avg: 4.11 |  53,689 ratings |  Drama|Romance



Get Recommendations (LSH Cold-Start)

UI App Screenshots

Your Personalized Recommendations

Generated using LSH-based cold-start algorithm

1. Terminator, The (1984)

Genres: Action|Sci-Fi|Thriller

Match Score: 23.8%

 Based on LSH approximate nearest neighbor search

Avg Rating

3.90 ★

 38,632 ratings

2. Terminator 3: Rise of the Machines (2003)

Genres: Action|Adventure|Sci-Fi

Match Score: 8.7%

 Based on LSH approximate nearest neighbor search

Avg Rating

3.06 ★

 11,546 ratings

3. Abyss, The (1989)

Genres: Action|Adventure|Sci-Fi|Thriller

Match Score: 4.4%

 Based on LSH approximate nearest neighbor search

Avg Rating

3.67 ★

 17,568 ratings

4. Predator (1987)

Genres: Action|Sci-Fi|Thriller

Match Score: 1.9%

 Based on LSH approximate nearest neighbor search

Avg Rating

3.66 ★

 16,762 ratings

5. Close Encounters of the Third Kind (1977)

Genres: Adventure|Drama|Sci-Fi

Match Score: 1.9%

 Based on LSH approximate nearest neighbor search

Avg Rating

3.78 ★

 14,584 ratings



Queries